TÀI LIỆU CHI TIẾT KIẾN TRÚC TẬP LỆNH (ISA) VÀ CÁC TÍN HIỆU ĐIỀU KHIỂN VI XỬ LÝ RISC-V RV32I PIPELINE

---

1. Tổng Quan Kiến Trúc Vi Xử Lý

- **Kiến trúc Tập lệnh (ISA):** RISC-V RV32I (32-bit Integer Base Instruction Set). Độ rộng từ lệnh và đường dẫn dữ liệu (Datapath) là 32-bit.
- **Cấu trúc thực thi:** Pipeline 5 tầng (Instruction Fetch - IF, Instruction Decode - ID, Execute - EX, Memory Access - MEM, Write Back - WB).
- **Tập thanh ghi:** 32 thanh ghi đa dụng 32-bit (x0 đến x31), trong đó x0 luôn cố định bằng 0.
- **Bộ nhớ dữ liệu (Data Memory):** Dung lượng 4096 Bytes (4KB), tổ chức theo cấu trúc Little-Endian, hỗ trợ truy xuất theo Byte (8-bit), Half-word (16-bit) và Word (32-bit).

2. Định Dạng Lệnh 32-bit và Giải Mã Immediate

Trong kiến trúc RV32I, lệnh có độ rộng 32-bit (Instr[31:0]) với 7 bit thấp nhất Instr[6:0] luôn là trường **Opcode**. Trường địa chỉ thanh ghi đích rd nằm ở Instr[11:7], thanh ghi nguồn 1 rs1 ở Instr[19:15], thanh ghi nguồn 2 rs2 ở Instr[24:20], func3 ở Instr[14:12] và func7 ở Instr[31:25]. Khối Extend.v nhận Instr[31:7] và tín hiệu điều khiển 3-bit ImmSrc_o (src) để tạo ra hằng số mở rộng 32-bit imm_extd:

Mã ImmSrc_o	Định dạng lệnh	Công thức ghép bit và mở rộng dấu (32-bit imm_extd)	Ý nghĩa
3'b001	**I-Type** (Tính toán, Load, JALR)	{{20{Instr[31]}}, Instr[31:20]}	Mở rộng dấu hằng số 12-bit có dấu.
3'b010	**S-Type** (Store: SB, SH, SW)	{{20{Instr[31]}}, Instr[31:25], Instr[11:7]}	Ghép 2 phần offset 12-bit và mở rộng dấu.
3'b011	**B-Type** (Branch)	{{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0}	Offset 13-bit (bit 0 luôn bằng 0) cho địa chỉ nhảy tương đối.
3'b100	**U-Type** (LUI, AUIPC)	{Instr[31:12], 12'b0}	Đưa 20-bit hằng số lên 20-bit cao, 12-bit thấp điền 0.
3'b101	**J-Type** (JAL)	{{12{Instr[31]}}, Instr[19:12], Instr[20], Instr[30:21], 1'b0}	Offset 21-bit (bit 0 luôn bằng 0) mở rộng dấu cho lệnh nhảy JAL.
3'b110	**I-Type Shift** (SLLI, SRLI, SRAI)	{27'b0, Instr[24:20]}	Trích xuất 5-bit lượng dịch (shamt) không dấu từ Instr[24:20].

3. Khối Điều Khiển Chính (Module Ctrl_Unit.v)

3.1. Giải thích ý nghĩa các tín hiệu điều khiển

- RegWrite_o (1-bit): Cho phép ghi kết quả vào Tập thanh ghi (1 = Cho phép ghi, 0 = Không ghi).
- ResultSrc_o (2-bit): Chọn nguồn dữ liệu để ghi ngược về thanh ghi ở tầng WB:
    - 2'b00: Kết quả từ khối tính toán ALU.

    - 2'b01: Dữ liệu đọc ra từ bộ nhớ Data_Mem (lệnh Load).
    - 2'b10: Địa chỉ lệnh kế tiếp PC + 4 (lưu địa chỉ quay về cho lệnh JAL, JALR).
    - 2'b11: Giá trị từ đường tính toán U-Type (imm_extd cho LUI hoặc PC + imm_extd cho AUIPC).
- MemWrite_o (4-bit): Tín hiệu điều khiển ghi vào Data Memory (0000 = Không ghi, 0001 = SB, 0010 = SH, 1000 = SW).
- MemRead_o (4-bit): Tín hiệu điều khiển đọc từ Data Memory (0000 = Không đọc, 0001 = LB, 0010 = LH, 1000 = LW, 1001 = LBU, 1010 = LHU).
- Jump_o (1-bit): Bật lên 1 khi gặp lệnh nhảy không điều kiện (JAL, JALR).
- Jalr_o (1-bit): Bật lên 1 riêng cho lệnh JALR (để chọn địa chỉ đích từ RS1 + Imm thay vì PC + Imm).
- Branch_o (1-bit): Bật lên 1 khi gặp lệnh rẽ nhánh có điều kiện (B-Type). Kết hợp với ALU_Br qua cổng AND để tạo tín hiệu chuyển hướng PC.
- ALUsrc_o (1-bit): Chọn toán hạng thứ 2 cho ALU (0 = Dữ liệu từ thanh ghi RT_data / RS2, 1 = Hằng số mở rộng imm_extd).
- ALUop_o (2-bit): Mã phân loại nhóm phép toán gửi sang ALUctrl.v (00 = Load/Store, 01 = Branch, 10 = R-Type / I-Type tính toán).
- ImmSrc_o (3-bit): Điều khiển kiểu giải mã hằng số ở module Extend.v.
- ImmUI_o (1-bit): Phân biệt giữa 2 lệnh U-Type (1 = LUI lấy trực tiếp hằng số imm_extd; 0 = AUIPC lấy pc_target = PC + imm_extd).

3.2. Bảng tổng hợp tín hiệu điều khiển theo Opcode

Nhóm Lệnh	Opcode (opcode_i)	Reg Write	ResultSrc	Mem Write	Mem Read	Jump	Jalr	Branch	ALUsrc	ALUop	ImmSrc	ImmUI
R-Type (ADD,	011_00	1	00	0000	0000	0	0	0	0	10	xxx	x

Nhóm Lệnh	Opcode (opcode_i)	RegWrite	ResultSrc	MemWrite	MemRead	Jump	Jalr	Branch	ALUsrc	ALUop	ImmSrc	ImmUI
SUB, XOR, OR, AND, SLL, SRL, SRA, SLT, SLTU)	11											
I-Type Tính toán (ADDI, XORI, ORI, ANDI, SLTI, SLTIU)	001_0011	1	00	0000	0000	0	0	0	1	10	001	x
I-Type Dịch bit (SLLI, SRLI, SRAI - func3= 001, 101)	001_0011	1	00	0000	0000	0	0	0	1	10	110	x

Nhóm Lệnh	Opcode (opcode_i)	RegWrite	ResultSrc	MemWrite	MemRead	Jump	Jalr	Branch	ALUSrc	ALUop	ImmSrc	ImmUI
I-Type Load (LB, LH, LW, LBU, LHU)	000_0011	1	01	0000	Theo func3	0	0	0	1	00	001	x
S-Type Store (SB, SH, SW)	010_0011	0	xx	Theo func3	0000	0	0	0	1	00	010	x
B-Type Branch (BEQ, BNE, BLT, BGE, BLTU, BGEU)	110_0011	0	xx	0000	0000	0	0	1	0	01	011	x
J-Type (JAL)	110_1111	1	10	0000	0000	1	0	0	x	xx	101	x

Nhóm Lệnh	Opcode (opcode_i)	Reg Write	ResultSrc	Mem Write	Mem Read	Jump	Jalr	Branch	ALUsrc	ALUop	ImmSrc	ImmUI
I-Type Jump (JALR)	110_0111	1	10	0000	0000	1	1	0	x	xx	001	x
U-Type (LUI)	011_0111	1	11	0000	0000	0	0	0	x	xx	100	1
U-Type (AUIPC)	001_0111	1	11	0000	0000	0	0	0	x	xx	100	0

4. Chi Tiết Giải Mã ALU (Module ALUctrl.v và ALU.v)

Module ALUctrl.v kết hợp ALUop_i (2-bit), opcode_i (bit số 5 của opcode: Instr[5]), func3_i (Instr[14:12]) và func7_i (bit số 5 của func7: Instr[30]) để sinh ra mã điều khiển 4-bit ALUctrl_o gửi tới ALU.v:

Tên Lệnh	ALUop_i	opcode[5]	func3_i	func7[5]	Mã ALUctrl_o	Hoạt động trong ALU.v (result / ALU_Br)
LB, LH, LW, LBU, LHU	00	0	000, 001, 010, 100, 101	x	4'h0 (ADD)	result = data1_i + data2_i (Tính địa chỉ đọc

Tên Lệnh	ALUop_i	opcode[5]	func3_i	func7[5]	Mã ALUctrl_o	Hoạt động trong ALU.v (result / ALU_Br)
						bộ nhớ)
SB, SH, SW	00	1	000, 001, 010	x	4'h0 (ADD)	result = data1_i + data2_i (Tính địa chỉ ghi bộ nhớ)
ADD / ADDI	10	1 / 0	000	0 / x	4'h0 (ADD)	result = data1_i + data2_i
SUB	10	1	000	1	4'h1 (SUB)	result = data1_i - data2_i
XOR / XORI	10	1 / 0	100	0 / x	4'h2 (XOR)	result = data1_i ^ data2_i
OR / ORI	10	1 / 0	110	0 / x	4'h3 (OR)	result = data1_i | data2_i
AND / ANDI	10	1 / 0	111	0 / x	4'h4 (AND)	result = data1_i & data2_i
SLL / SLLI	10	1 / 0	001	0	4'h5 (SLL)	result = data1_i << data2_i[4:0]

Tên Lệnh	ALUop_i	opcode[5]	func3_i	func7[5]	Mã ALUctrl_o	Hoạt động trong ALU.v (result / ALU_Br)
						(Dịch trái logic)
SRL / SRLI	10	1 / 0	101	0	4'h6 (SRL)	result = data1_i >> data2_i[4:0] (Dịch phải logic)
SRA / SRAI	10	1 / 0	101	1	4'h7 (SRA)	result = $signed(data1_i) >>> data2_i[4:0] (Dịch phải số học)
SLT / SLTI	10	1 / 0	010	0 / x	4'h8 (SLT)	result = ($signed(data1_i) < $signed(data2_i)) ? 1 : 0
SLTU / SLTIU	10	1 / 0	011	0 / x	4'h9 (SLTU)	result = (data1_i < data2_i) ? 1 : 0
BEQ	01	x	000	x	4'hA (BEQ)	ALU_Br = (data1_i == data2_i)

Tên Lệnh	ALUop_i	opcode[5]	func3_i	func7[5]	Mã ALUctrl_o	Hoạt động trong ALU.v (result / ALU_Br)
BNE	01	x	001	x	4'hB (BNE)	ALU_Br = (data1_i != data2_i)
BLT	01	x	100	x	4'hC (BLT)	ALU_Br = ($signed(data1_i) < $signed(data2_i))
BGE	01	x	101	x	4'hD (BGE)	ALU_Br = ~($signed(data1_i) < $signed(data2_i))
BLTU	01	x	110	x	4'hE (BLTU)	ALU_Br = (data1_i < data2_i)
BGEU	01	x	111	x	4'hF (BGEU)	ALU_Br = ~(data1_i < data2_i)

5. Tín Hiệu Điều Khiển Bộ Nhớ Dữ Liệu (Module Data_Mem.v)

Lệnh	func3	Mã rEN (MemRead_o)	Mã wEN (MemWrite_o)	Thao tác phần cứng trên mảng DATA_MEM
LB	000	4'b0001	4'b0000	Đọc 1 byte tại addr_i, mở rộng dấu bit 7 ra 24-bit cao: {{24{DATA_MEM[addr_i][7]}}, DATA_MEM[addr_i]}
LH	001	4'b0010	4'b0000	Đọc 2 byte Little-Endian, mở rộng dấu bit 7 của byte cao ra 16-bit cao: {{16{DATA_MEM[addr_i+1][7]}}, DATA_MEM[addr_i+1], DATA_MEM[addr_i]}
LW	010	4'b1000	4'b0000	Đọc 4 byte liên tiếp: {DATA_MEM[addr_i+3], DATA_MEM[addr_i+2], DATA_MEM[addr_i+1], DATA_MEM[addr_i]}
LBU	100	4'b1001	4'b0000	Đọc 1 byte không dấu, điền 0 vào 24-bit cao: {24'b0, DATA_MEM[addr_i]}

Lệnh	func3	Mã rEN (MemRead_o)	Mã wEN (MemWrite_o)	Thao tác phần cứng trên mảng DATA_MEM
LHU	101	4'b1010	4'b0000	Đọc 2 byte không dấu, điền 0 vào 16-bit cao: {16'b0, DATA_MEM[addr_i+1], DATA_MEM[addr_i]}
SB	000	4'b0000	4'b0001	Kiểm tra wEN[0]: Ghi data_i[7:0] vào DATA_MEM[addr_i]
SH	001	4'b0000	4'b0010	Kiểm tra wEN[1]: Ghi 2 byte thấp data_i[15:0] vào addr_i và addr_i+1
SW	010	4'b0000	4'b1000	Kiểm tra wEN[3]: Ghi cả 4 byte data_i[31:0] vào addr_i đến addr_i+3

6. Khối Xử Lý Xung Đột Đường Ống (Module Hazard_Unit.v)

Trong thiết kế Pipeline, khối Hazard_Unit.v theo dõi địa chỉ các thanh ghi nguồn/đích ở các tầng Decode (D), Execute (E), Memory (M) và Write Back (W) để phát sinh 3 nhóm tín hiệu xử lý:

6.1. Tín hiệu Chuyển tiếp dữ liệu (Data Forwarding: Forward1E_o, Forward2E_o)

Giải quyết xung đột đọc-sau-ghi (RAW Data Hazard) bằng cách lấy kết quả từ tầng MEM hoặc WB đưa thẳng về ngõ vào của ALU tại tầng EX:

- 2'b10 (Forward từ tầng MEM): Khi (RS_addrE_i == RD_addrM_i) & RegWriteM_i & (RS_addrE_i != 0). Ưu tiên cao nhất vì dữ liệu ở tầng MEM mới hơn tầng WB.
- 2'b01 (Forward từ tầng WB): Khi (RS_addrE_i == RD_addrW_i) & RegWriteW_i & (RS_addrE_i != 0).
- 2'b00 (Không Forward): Lấy dữ liệu bình thường từ thanh ghi đường ống ID_EX.

(Quy tắc tương tự áp dụng cho toán hạng thứ 2 RT_addrE_i với tín hiệu Forward2E_o).

6.2. Tín hiệu Tạm dừng đường ống (Pipeline Stall: lwStall, StallF_o, StallD_o)

- Tín hiệu nội bộ lwStall bật lên 1 khi lệnh đang ở tầng EX là lệnh **Load** (ResultSrcE_i == 2'b01) hoặc lệnh **U-Type LUI/AUIPC** (ResultSrcE_i == 2'b11) VÀ thanh ghi đích của nó trùng với thanh ghi nguồn của lệnh kế tiếp đang ở tầng ID: (RS_addrD_i == RD_addrE_i) | (RT_addrD_i == RD_addrE_i).
- StallF_o = lwStall: Đóng băng thanh ghi PC (giữ nguyên địa chỉ lệnh hiện tại).
- StallD_o = lwStall: Đóng băng thanh ghi đường ống IF_ID.

6.3. Tín hiệu Xóa đường ống (Pipeline Flush: FlushD_o, FlushE_o)

- FlushD_o = PC_SrcE_i: Khi có lệnh rẽ nhánh hoặc nhảy (Branch Taken hoặc Jump tại tầng EX làm PC_SrcE_i = 1), xóa lệnh sai đã nạp vào thanh ghi IF_ID.
- FlushE_o = lwStall | PC_SrcE_i: Đưa tín hiệu vào chân CLR của thanh ghi ID_EX để chèn lệnh rỗng (NOP - xóa toàn bộ tín hiệu điều khiển về 0) khi xảy ra Stall hoặc khi rẽ nhánh thành công.